



Proceso de Desarrollo de Software

3.4.210

Clase 8

Jonathan Leonel Pepe

Patrón de diseño

- El concepto de ***patrón de diseño*** ya fue abordado innumerables veces en el curso, aún así...
- Son **soluciones** probadas a **problemas comunes** en el diseño de software orientado a objetos.
 - No son código listo para copiar y pegar.
 - Tienen estructura definida (*nombre, problema, solución, consecuencias*).
 - Buscan mejorar la reutilización, flexibilidad y mantenibilidad del software.

Son herramientas esenciales en la *mochila* de cualquier desarrollador de software.

Patrón vs Principio

- Un **Principio** es una guía general y abstracta que orienta cómo deberíamos diseñar el software.
 - Son enunciados amplios sobre qué debemos evitar o lograr.
 - *Ejemplos: SRP, OCP, Tell, Don't Ask.*
- Un **Patrón** es una solución probada y reutilizable para un problema recurrente en un contexto específico.
 - Tienen estructura definida.
 - *Ejemplos: Singleton, Decorator, Strategy.*



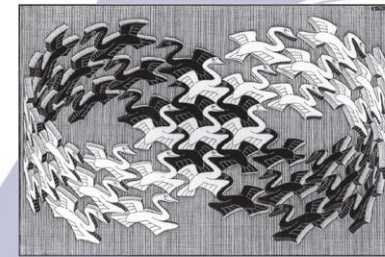
Patrones GoF

- Los **patrones GoF** son los **23 patrones** de diseño de software recopilados y popularizados en el libro “*Design Patterns: Elements of Reusable Object-Oriented Software*” (1994).
- El libro fue escrito por cuatro autores: *Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides*, conocidos como la “**Gang of Four (GoF)**”.

Design Patterns

Elements of Reusable
Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch

Patrones GoF

Clasificación

- Los **patrones GoF** se clasifican en tres grandes categorías:
 - **Creacionales**
 - Se centran en la creación de objetos, permitiendo un mayor control sobre el proceso de instanciación.
 - **Estructurales**
 - Se centran en cómo se relacionan las clases y los objetos para formar estructuras más grandes.
 - **Comportamiento**
 - Se centran en la comunicación entre objetos, especialmente en términos de responsabilidades.

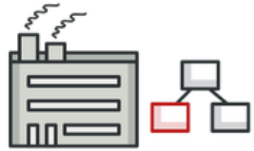
Patrones GoF

Clasificación

THE 23 GANG OF FOUR DESIGN PATTERNS

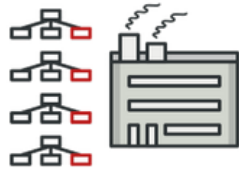
C	Abstract Factory	S	Facade	S	Proxy
S	Adapter	C	Factory Method	B	Observer
S	Bridge	S	Flyweight	C	Singleton
C	Builder	B	Interpreter	B	State
B	Chain of Responsibility	B	Iterator	B	Strategy
B	Command	B	Mediator	B	Template Method
S	Composite	B	Memento	B	Visitor
S	Decorator	C	Prototype		

Patrones creacionales



Factory Method

Proporciona una interfaz para la creación de objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.



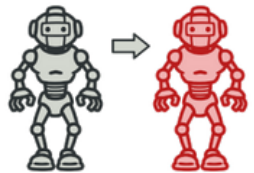
Abstract Factory

Permite producir familias de objetos relacionados sin especificar sus clases concretas.



Builder

Permite construir objetos complejos paso a paso. Este patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.



Prototype

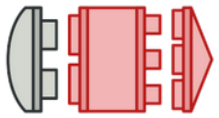
Permite copiar objetos existentes sin que el código dependa de sus clases.



Singleton

Permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.

Patrones estructurales



Adapter

Permite la colaboración entre objetos con interfaces incompatibles.



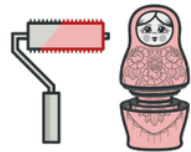
Bridge

Permite dividir una clase grande o un grupo de clases estrechamente relacionadas, en dos jerarquías separadas (abstracción e implementación) que pueden desarrollarse independientemente la una de la otra.



Composite

Permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.



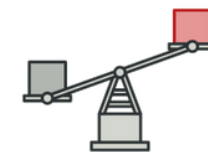
Decorator

Permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.



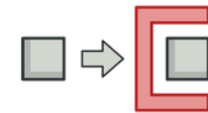
Facade

Proporciona una interfaz simplificada a una biblioteca, un framework o cualquier otro grupo complejo de clases.



Flyweight

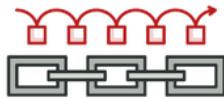
Permite mantener más objetos dentro de la cantidad disponible de memoria RAM compartiendo las partes comunes del estado entre varios objetos en lugar de mantener toda la información en cada objeto.



Proxy

Permite proporcionar un sustituto o marcador de posición para otro objeto. Un proxy controla el acceso al objeto original, permitiéndote hacer algo antes o después de que la solicitud llegue al objeto original.

Patrones de comportamiento



Chain of Responsibility

Permite pasar solicitudes a lo largo de una cadena de manejadores. Al recibir una solicitud, cada manejador decide si la procesa o si la pasa al siguiente manejador de la cadena.



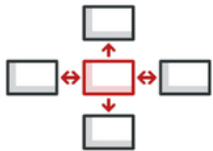
Command

Convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Esta transformación te permite parametrizar los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud y soportar operaciones que no se pueden realizar.



Iterator

Permite recorrer elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).



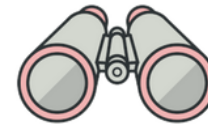
Mediator

Permite reducir las dependencias caóticas entre objetos. El patrón restringe las comunicaciones directas entre los objetos, forzándolos a colaborar únicamente a través de un objeto mediador.



Memento

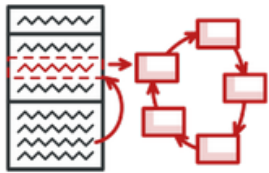
Permite guardar y restaurar el estado previo de un objeto sin revelar los detalles de su implementación.



Observer

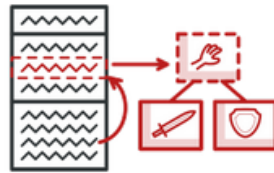
Permite definir un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.

Patrones de comportamiento



State

Permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiara su clase.



Strategy

Permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.



Template Method

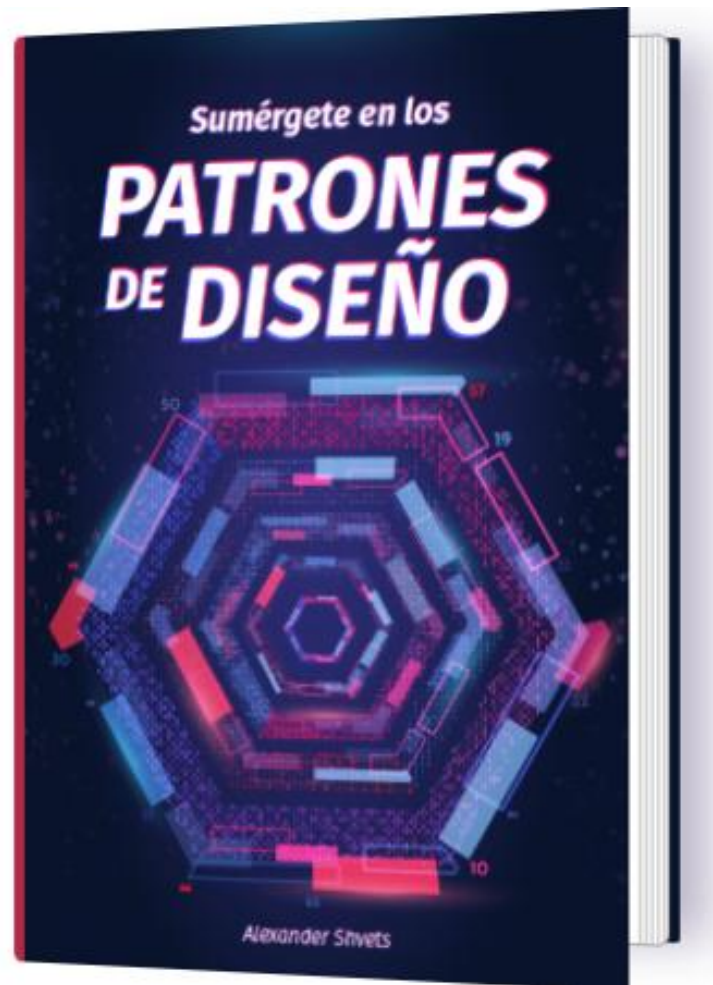
Define el esqueleto de un algoritmo en la superclase pero permite que las subclasses sobrescriban pasos del algoritmo sin cambiar su estructura.



Visitor

Permite separar algoritmos de los objetos sobre los que operan.

Bibliografía utilizada de aquí en adelante



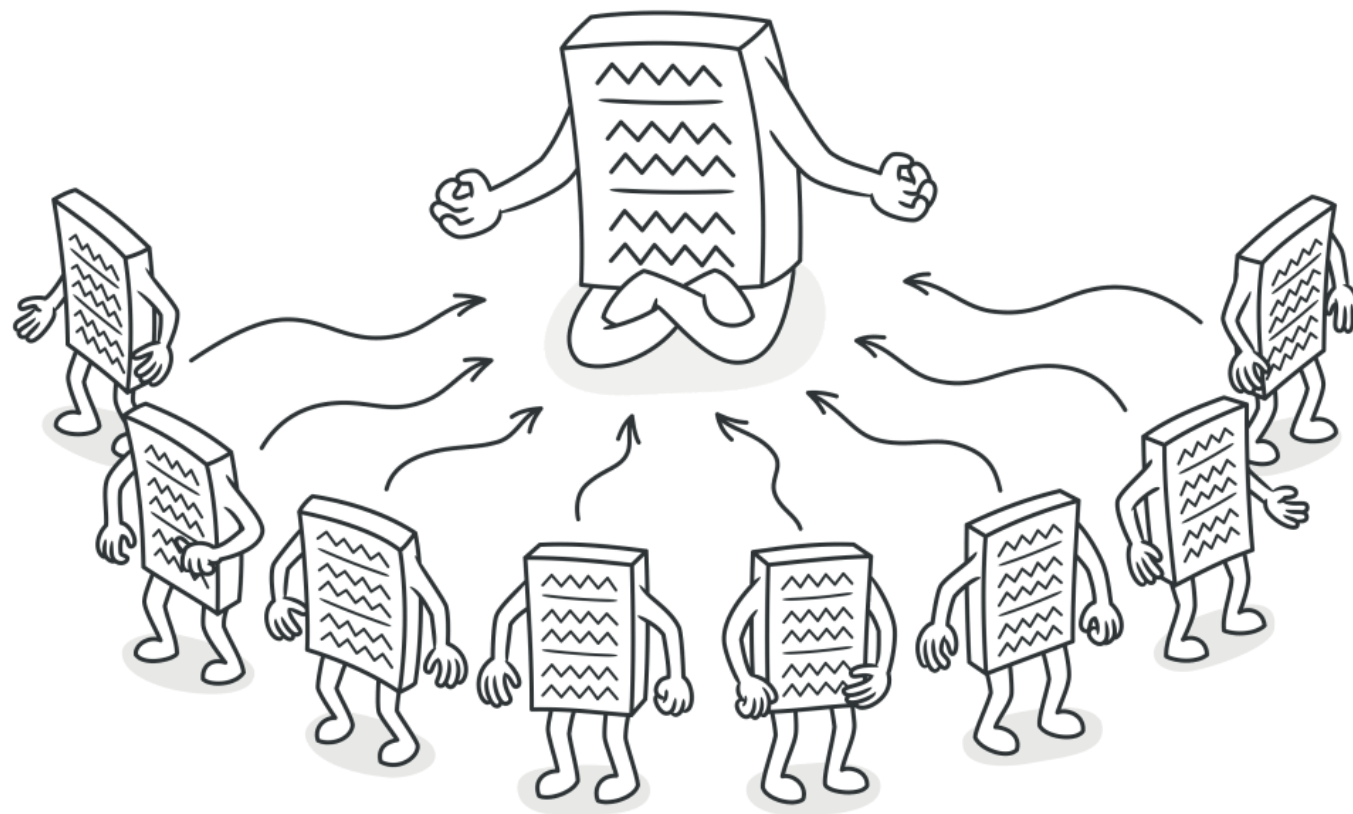


Singleton

Presentación

- **Singleton** es un patrón de diseño creacional que nos permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.
 - Imaginemos una agencia de gobierno como la *CIA*.
 - Por ley solo puede haber una.
 - No importa quién necesite interactuar con ella, siempre se hará a través de la misma y única instancia.

Singleton Presentación



Singleton Problema

Múltiples instancias innecesarias

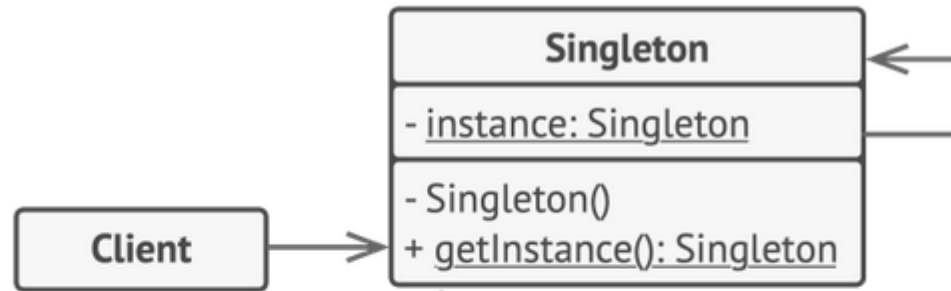
- Imaginemos que tenemos una clase para gestionar la configuración de nuestra aplicación o la conexión a una base de datos.
- Si permitimos que se creen múltiples objetos de esta clase podríamos enfrentar los siguientes problemas:
 - Varias instancias podrían estar modificando la misma información al mismo tiempo.
 - Crear y mantener múltiples conexiones puede ser muy costoso en términos de memoria y tiempo de procesamiento.
 - Sin un punto central de control se nos hace difícil coordinar el estado de la aplicación de manera global.

Singleton Solución

Una única instancia controlada

- El patrón *Singleton* resuelve estos problemas al seguir dos principios clave:
 - **Restringir la creación:** El constructor de la clase se hace privado, lo que impide que otras clases creen instancias directamente usando *new*.
 - **Proveer un acceso global:** Se crea un método estático y público (*generalmente llamado getInstance()*) que se encarga de:
 - Verificar si ya existe una instancia.
 - Si no existe, la crea (*lazy initialization*).
 - Si ya existe, simplemente la devuelve.
- De esta manera, no importa quién o cuándo se solicite la instancia, siempre se obtendrá la misma.

Singleton Estructura

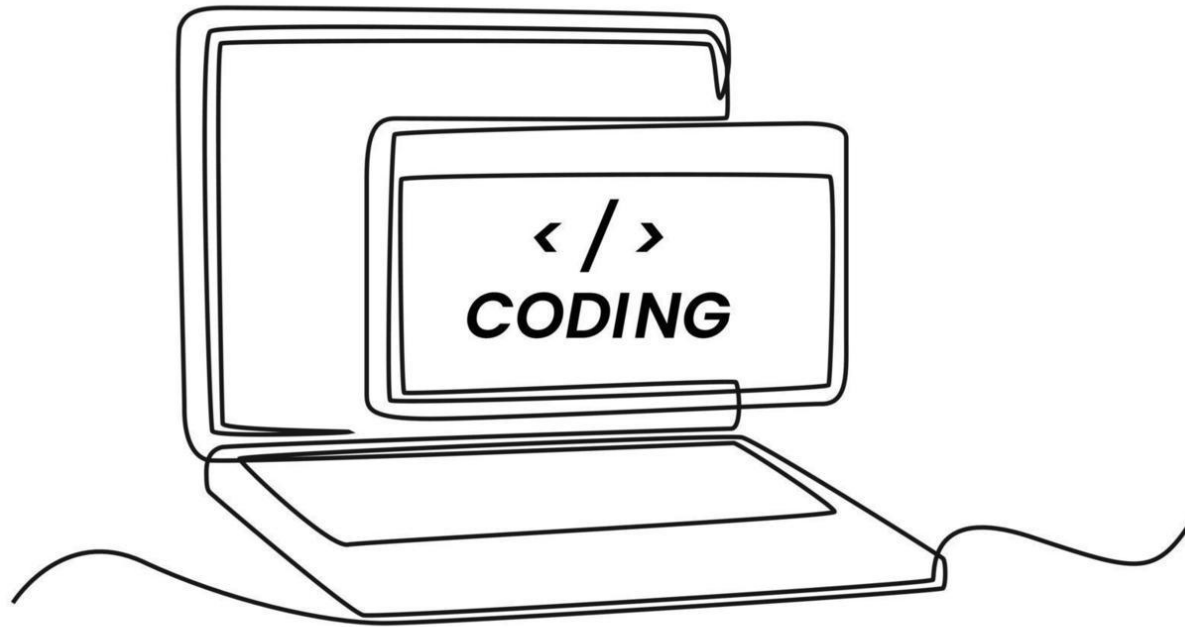


1 La clase **Singleton** declara el método estático `obtenerInstancia` que devuelve la misma instancia de su propia clase.

El constructor del Singleton debe ocultarse del código cliente. La llamada al método `obtenerInstancia` debe ser la única manera de obtener el objeto de Singleton.

```
if (instance == null) {
    // Nota: si estás creando una aplicación
    // que soporte el multihilo, debes
    // colocar un bloqueo de hilo aquí.
    instance = new Singleton()
}
return instance
```

Momento de programar

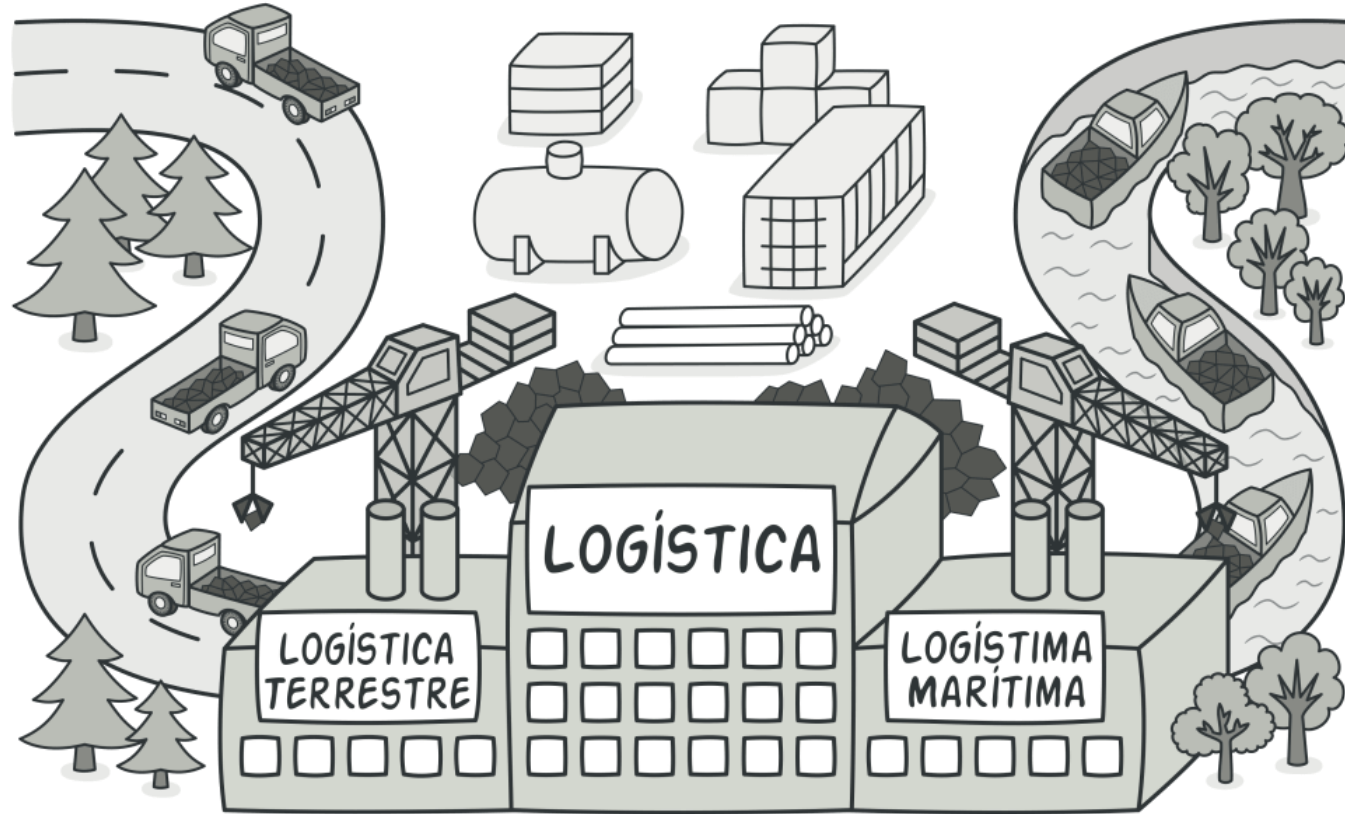


Factory Method

Presentación

- **Factory Method** es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.
 - Imaginemos una aplicación de logística para una empresa de transporte.
 - Si la aplicación solo transporta en camión, la clase *Truck* es suficiente.
 - Pero si un día la empresa decide usar barcos, necesitaríamos una nueva clase *Ship*.

Factory Method Presentación



Factory Method

Problema

Lógica de creación rígida

- Cuando creamos objetos directamente con el operador *new* dentro de nuestro código, nuestro programa se vuelve rígido y acoplado.
 - **Acoplamiento fuerte:** El código de nuestra aplicación se acopla directamente a las clases concretas (*Truck*, *Ship*).
 - **Violación del principio *abierto/cerrado*:** Añadir un nuevo tipo de transporte (*Ship*) requiere que modifiquemos la clase de la que se crea.

Factory Method Solución

Delegar la creación de objetos

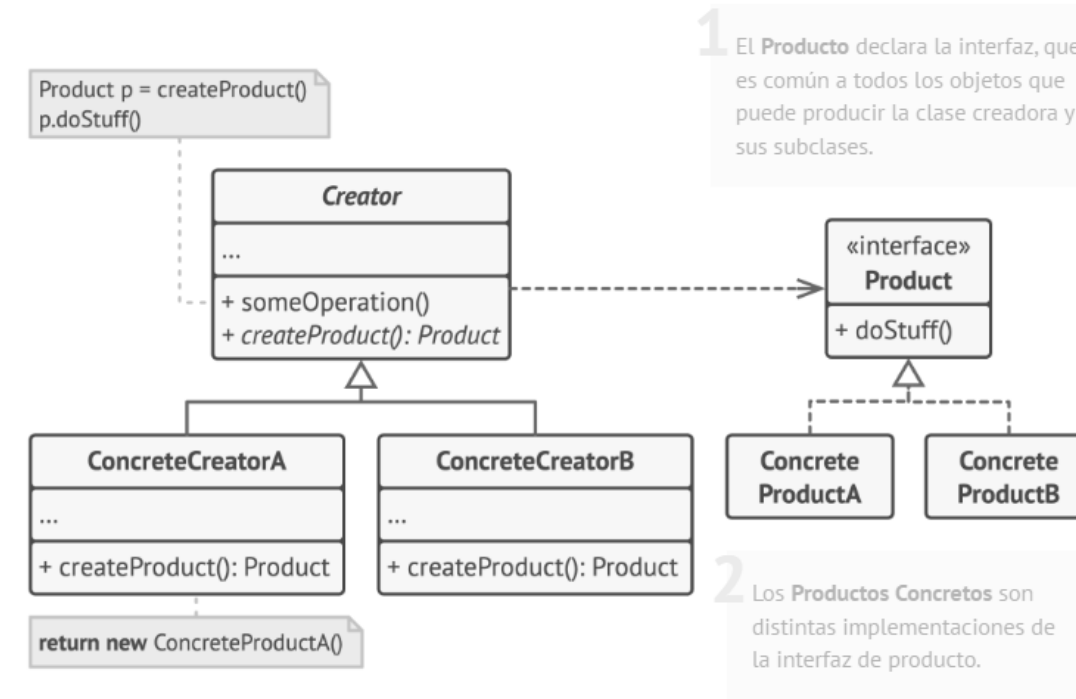
- El patrón *Factory Method* resuelve estos problemas al seguir dos principios clave:
 - **Método de fábrica:** Creamos una interfaz (*Creator*) con un método de fábrica abstracto (*factoryMethod*).
 - Este método se encarga de crear objetos.
 - **Subclases concretas:** Creamos subclases (*TruckCreator*, *ShipCreator*) que heredan de la interfaz y sobrescriben el método de fábrica para devolver el objeto concreto que necesitamos (*Truck*, *Ship*).
- De esta manera, el código que usa la fábrica (*Client*) trabaja con la interfaz de la clase (*Creator*), y no con las clases concretas.
 - Esto nos permite añadir nuevos tipos de transporte simplemente creando una nueva subclase (*AirplaneCreator*) sin tener que modificar el código que ya existe.

Factory Method Estructura

3 La clase **Creadora** declara el método fábrica que devuelve nuevos objetos de producto. Es importante que el tipo de retorno de este método coincida con la interfaz de producto.

Puedes declarar el patrón Factory Method como abstracto para forzar a todas las subclases a implementar sus propias versiones del método. Como alternativa, el método fábrica base puede devolver algún tipo de producto por defecto.

Observa que, a pesar de su nombre, la creación de producto **no** es la principal responsabilidad de la clase creadora. Normalmente, esta clase cuenta con alguna lógica de negocios central relacionada con los productos. El patrón Factory Method ayuda a desacoplar esta lógica de las clases concretas de producto. Aquí tienes una analogía: una gran empresa de desarrollo de software puede contar con un departamento de formación de programadores. Sin embargo, la principal función de la empresa sigue siendo escribir código, no preparar programadores.



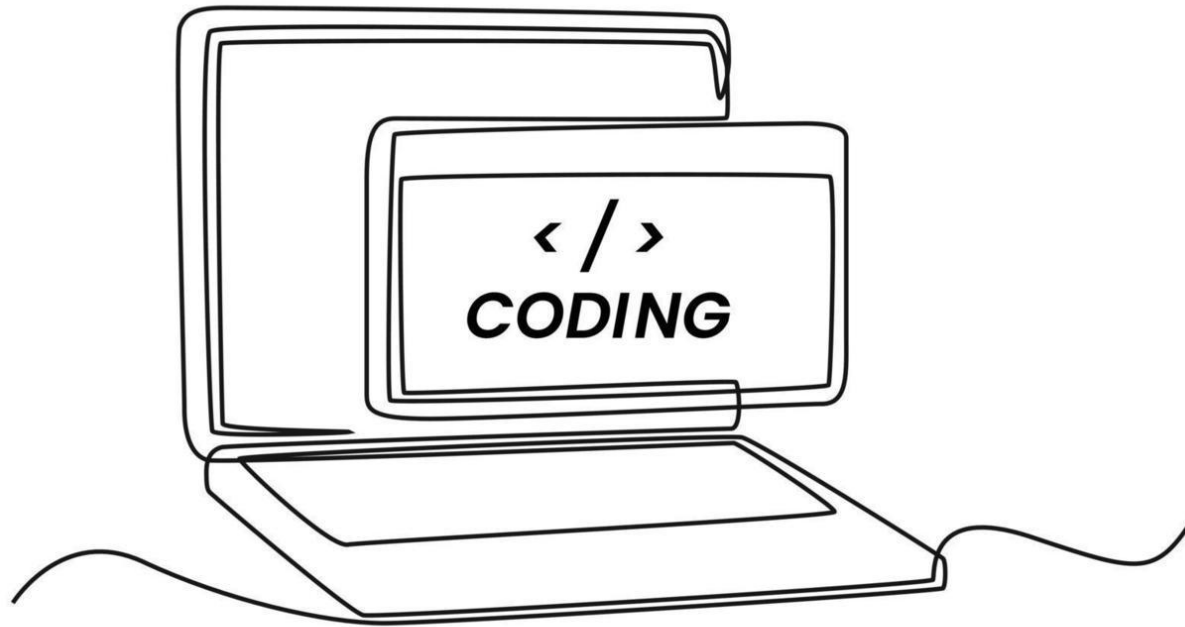
1 El **Producto** declara la interfaz, que es común a todos los objetos que puede producir la clase creadora y sus subclases.

2 Los **Productos Concretos** son distintas implementaciones de la interfaz de producto.

4 Los **Creadores Concretos** sobrescriben el Factory Method base, de modo que devuelva un tipo diferente de producto.

Observa que el método fábrica no tiene que **crear** nuevas instancias todo el tiempo. También puede devolver objetos existentes de una memoria caché, una agrupación de objetos, u otra fuente.

Momento de programar

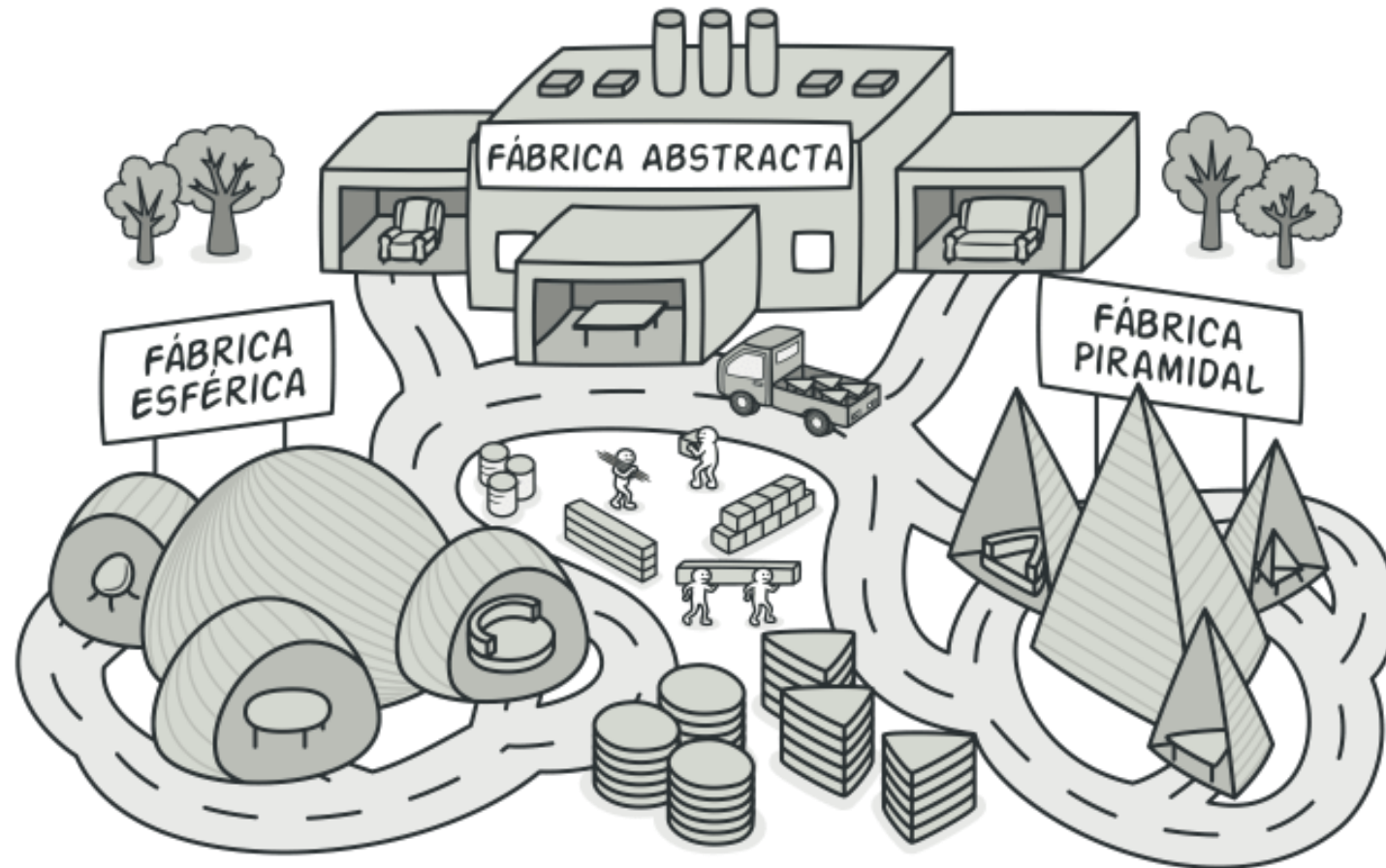


Abstract Factory

Presentación

- **Abstract Factory** es un patrón de diseño creacional que nos permite producir familias de objetos relacionados sin especificar sus clases concretas.
 - Imaginemos una empresa de muebles que produce productos de diferentes estilos, como *moderno* o *victoriano*.
 - Para cada estilo, la empresa debe fabricar *sillas*, *sofás* y *mesas* que coincidan.
 - En lugar de crear los objetos uno por uno, una fábrica abstracta puede garantizar que todos los productos que se crean pertenecen al mismo estilo.

Abstract Factory Presentación



Abstract Factory

Problema

Lógica de creación rígida

- Cuando nuestro código crea objetos directamente con el operador *new*, se vuelve rígido y difícil de cambiar.
- Si la empresa de muebles quisiera añadir un nuevo estilo, como *art deco*, tendríamos:
 - **Acoplamiento fuerte:** El código se acopla a las clases de los muebles concretos (*SillaModerna*, *SofaVictoriano*), lo que hace que sea muy difícil cambiar un estilo por otro.
 - **Violación del principio *abierto/cerrado*:** Si añadimos un nuevo estilo de muebles, debemos modificar el código existente para incluir la lógica de creación de los nuevos objetos.

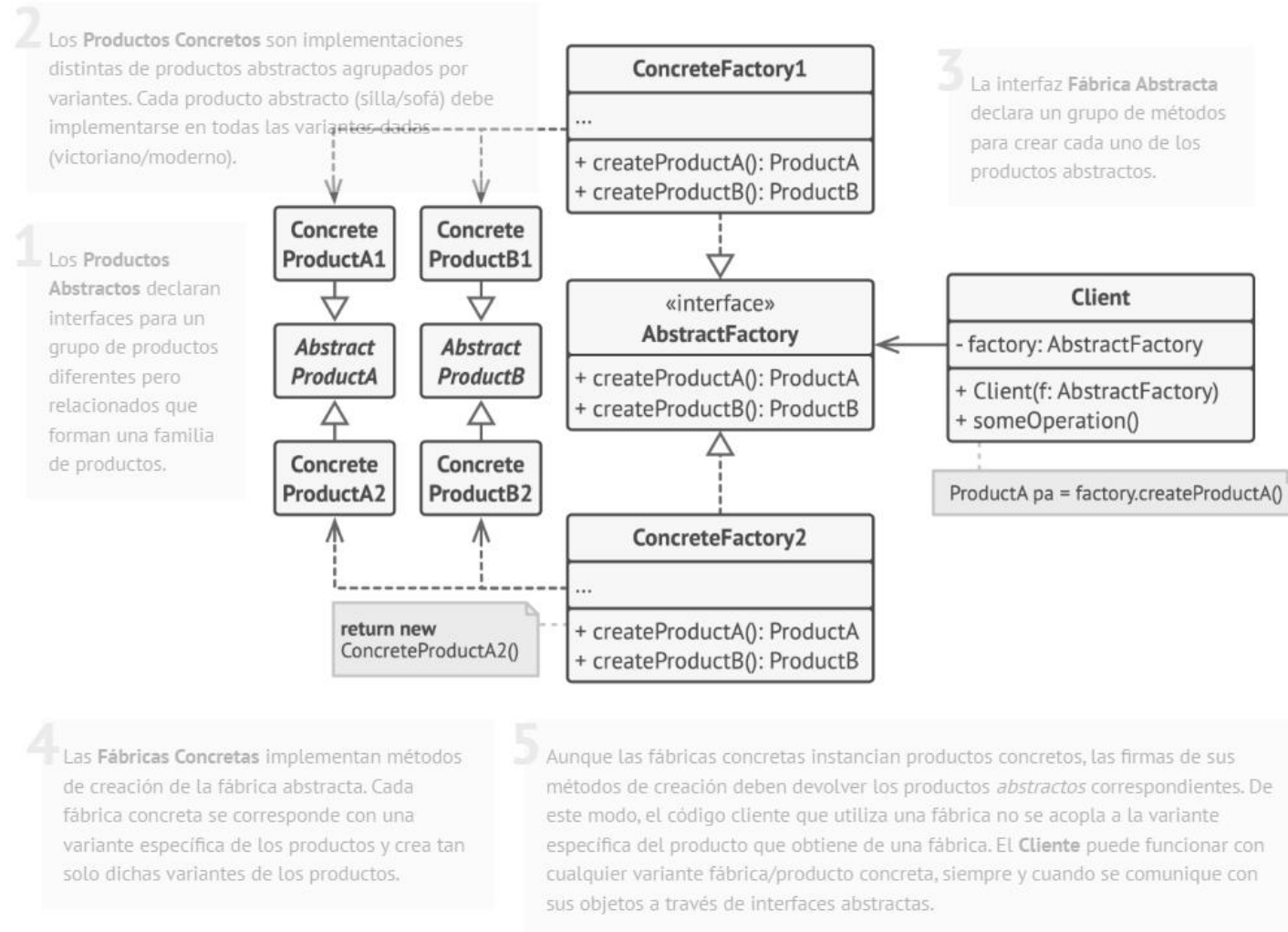
Abstract Factory Solución

Delegar la creación de familias de objetos

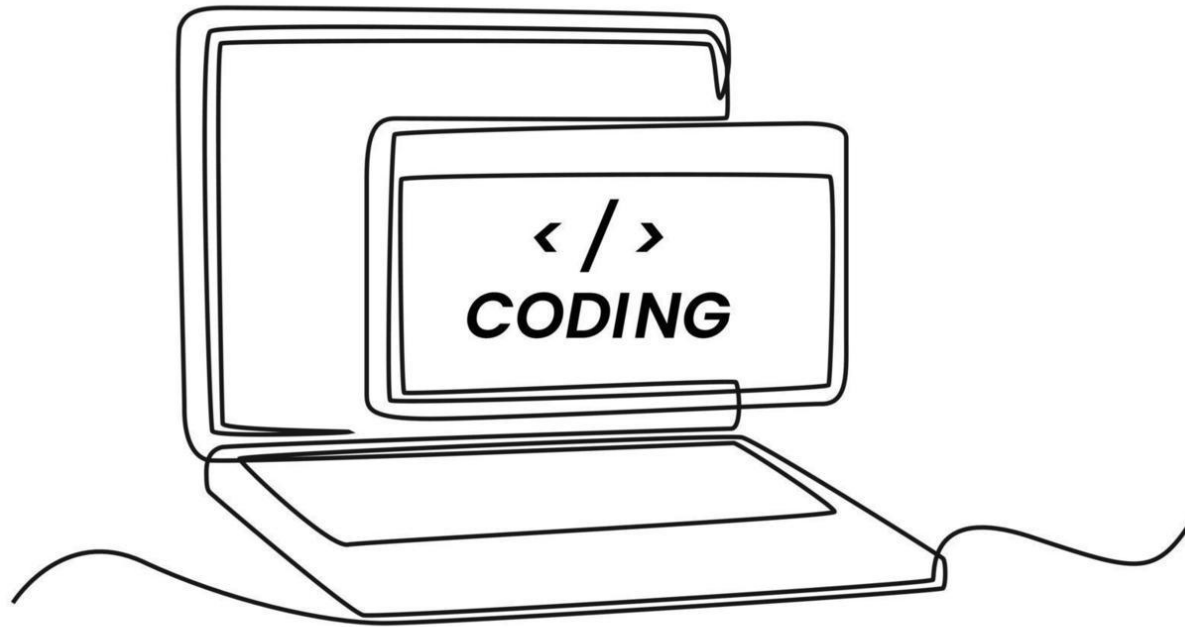
- El patrón *Abstract Factory* resuelve estos problemas al seguir dos principios clave:
 - **Define una fábrica abstracta:** Creamos una interfaz *FurnitureFactory* con métodos para crear cada producto de la familia (*createChair()*, *createSofa()*).
 - **Crea fábricas concretas:** Hacemos que cada estilo de muebles sea una fábrica concreta (*ModernFurnitureFactory*, *VictorianFurnitureFactory*) que implemente la interfaz y cree los objetos apropiados para ese estilo.
- De esta manera, el código de nuestro *cliente* solo usa la interfaz *FurnitureFactory*, sin saber con qué fábrica concreta está trabajando.
 - Esto nos permite cambiar de un estilo a otro (*ModernFurnitureFactory* a *VictorianFurnitureFactory*) simplemente cambiando la fábrica que se usa, sin modificar el código que la consume.

Abstract Factory

Estructura



Momento de programar



Homework

- Analizar los códigos presentados en clase.
 - Realizar modificaciones e introducir nuevas funcionalidades.
- Armar grupos para el *TIF*.
- Profundizar la lectura de **todos** los *patrones creacionales* y sus posibles aplicaciones en [Source Making](#).

